VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Natural Language Processing

Sentiment Analysis

Semester: 231

Advisors: Assoc. Prof Quan Thanh Tho
Student: Vo Hoang Nhat Khang - 2152646

HO CHI MINH CITY, NOVEMBER 2023

Contents

1 Introduction to sentiment analysis **1**

2 Models used in this assignment **1**
2.1 Convolutional Neural Networks . . . 1
2.2 Long Short-Term Memory . . . 3
2.3 K-Fold validation . . . 5
2.4 PhoBERT pre-trained model . . . 5
2.5 RoBERTa: An optimized BERT pretraining model . . . 6

3 Data **6**

4 Structure **10**
4.1 Using only CNN . . . 10
4.1.1 Define structure and arguments . . . 10
4.1.2 Train on training set . . . 11
4.1.3 Evaluation on test set . . . 12
4.2 Using only LSTM . . . 13
4.2.1 Defined structure and arguments . . . 13
4.2.2 Train on training set . . . 14
4.2.3 Evaluation on test set . . . 14
4.3 Using CNN & LSTM combination . . . 15
4.3.1 CNN layers before LSTM layers . . . 15
4.3.2 LSTM layers before CNN layers . . . 18
4.4 Applying K-Fold with PhoBERT . . . 20
4.4.1 Data preprocessing and folding data . . . 20
4.4.2 Training on train set . . . 22
4.4.3 Evaluate on test set . . . 23
4.5 RoBERTa combining with LSTM (in developing mode) . . . 24
4.5.1 Defined structure . . . 24
4.5.2 Trainer . . . 25
4.5.3 Configuring environment: . . . 25
4.5.4 Important parameters . . . 25
4.5.5 Training result . . . 26
4.5.6 Testing result . . . 26

5 Result and analysis **26**

6 Conclusion **27**

7 Reference **27**

1 Introduction to sentiment analysis

Sentiment analysis, also known as opinion mining, is a branch of natural language processing (NLP) that involves the use of computational methods to determine and interpret the emotional tone and subjective opinions expressed in textual data. In the era of vast digital communication, where opinions and sentiments are prolifically shared on social media, product reviews, news articles, and various online platforms, sentiment analysis plays a crucial role in understanding public sentiment, customer feedback, and overall attitudes towards different entities.

The primary objective of sentiment analysis is to automate the process of extracting insights from text data by classifying it as positive, negative, or neutral, based on the sentiments expressed. This technology leverages machine learning algorithms and linguistic rules to discern the emotional content within written or spoken language, enabling businesses, researchers, and organizations to gain valuable insights into public perception, make data-driven decisions, and enhance their overall understanding of user sentiment.

As the digital landscape continues to evolve, sentiment analysis remains a powerful tool with applications ranging from brand monitoring and customer feedback analysis to political polling and market research. Its ability to process and analyze vast amounts of textual data in real-time makes sentiment analysis an invaluable asset in today's information-driven world. This introduction sets the stage for a deeper exploration of the methodologies, applications, and challenges associated with sentiment analysis.

2 Models used in this assignment

2.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a class of deep learning algorithms designed for processing structured grid data, such as images and, more recently, sequences of data in natural language processing (NLP) tasks. Originally developed for computer vision applications, CNNs have proven to be highly effective in feature extraction and pattern recognition.

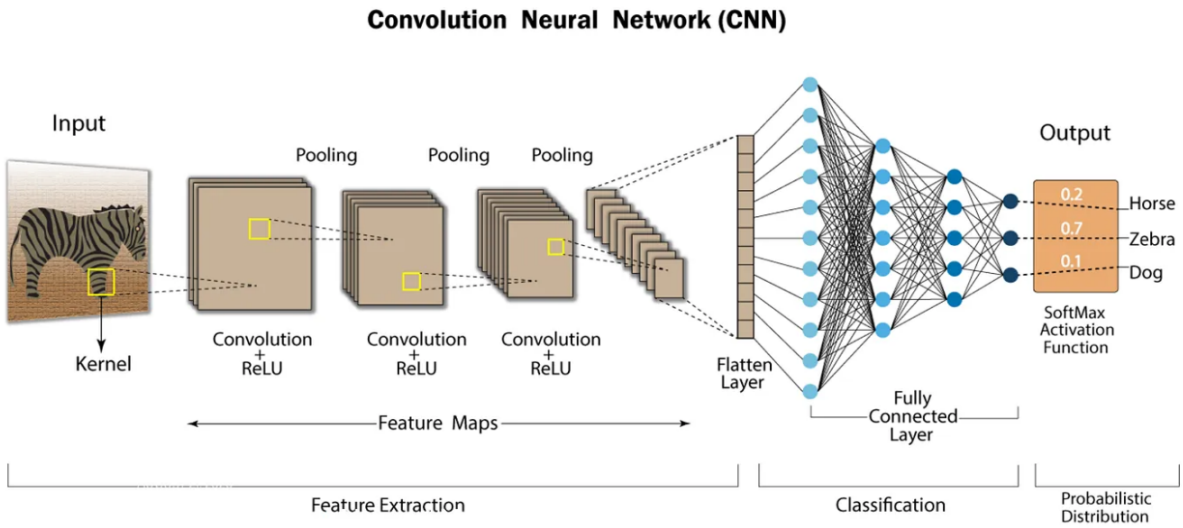

Figure 1: CNN used in NLP

Some key components of CNNs can be represented as follow:

1. **Convolutional layers:** The core operation in CNNs is convolution. Convolutional layers use filters (also called kernels) to slide over input data, capturing local patterns and features. This enables the network to learn hierarchical representations, from simple edges and textures to complex structures.

2. **Pooling layers:** Pooling layers downsample the spatial dimensions of the input, reducing computational complexity and providing a form of translation invariance. Common pooling operations include max pooling, which retains the maximum value in a local region, and average pooling, which computes the average.

3. **Activation functions:** Non-linear activation functions, such as ReLU (Rectified Linear Unit), introduce non-linearity to the model, allowing it to learn more complex relationships in the data.

4. **Fully connected layers:** These layers connect every neuron in one layer to every neuron in the next layer, forming the final output. Fully connected layers are typically used in the later stages of CNNs to combine extracted features and make predictions.

While initially developed for image processing, CNNs have been successfully applied to NLP tasks, taking advantage of their ability to capture local patterns. In NLP, the input data is often represented as word embeddings or character embeddings. The following are common use cases for CNNs in NLP:

- **Text Classification:**

  CNNs can be used for sentiment analysis, topic classification, and spam detection by learning hierarchical features from word embeddings.

- **Named Entity Recognition (NER):**

  Identifying entities (e.g., names of people, organizations) in text is a crucial NLP task. CNNs can be employed to capture local patterns for effective entity recognition.

- **Text Generation:**

  CNNs can contribute to language modeling and text generation by learning hierarchical representations of input sequences.

- **Document Classification:**

  For tasks like identifying the category of a document, CNNs can be applied to extract features from word embeddings and make predictions.

- **Semantic Role Labeling (SRL):**

  CNNs are utilized for extracting features and identifying semantic roles in sentences, aiding in understanding the relationships between different components of a sentence.

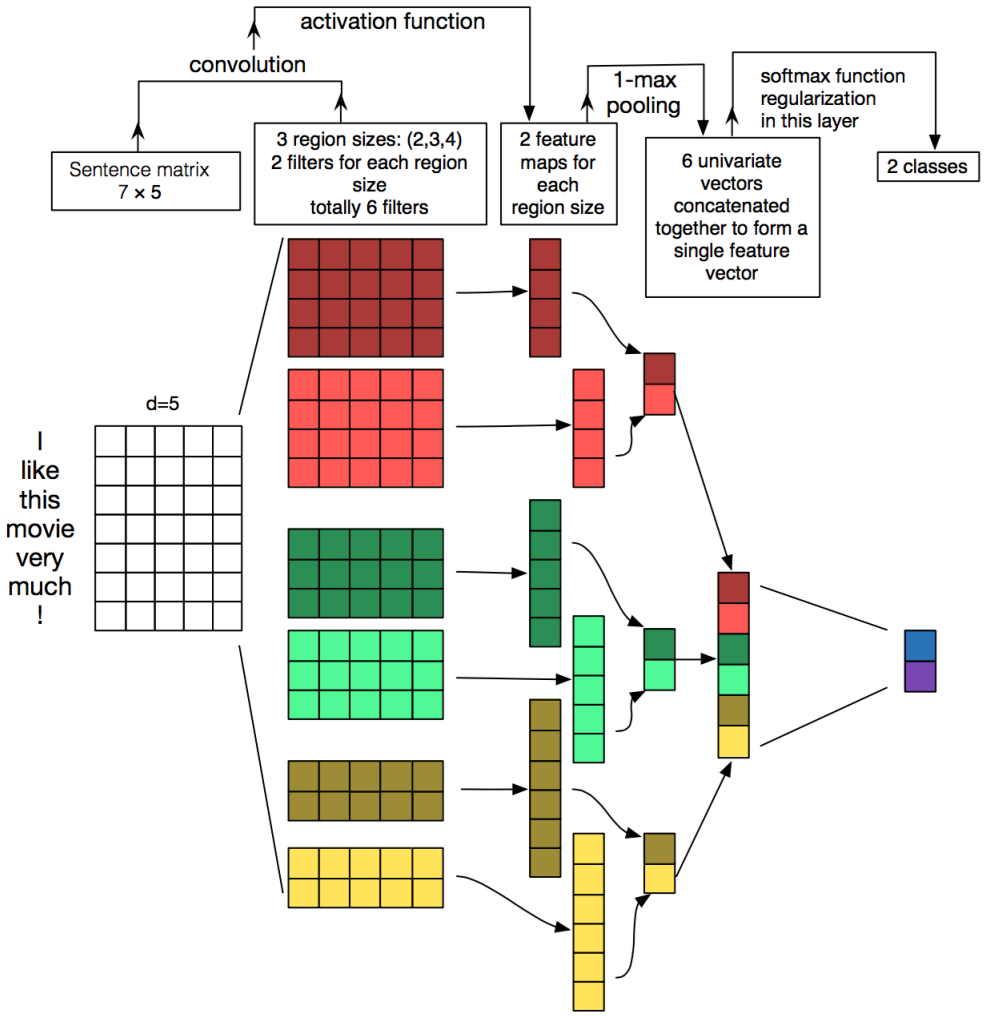

Figure 2: CNN used in NLP

2.2 Long Short-Term Memory

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) architecture designed to address the vanishing gradient problem, which can hinder the learning of long-range dependencies in sequential data. Introduced by Hochreiter and Schmidhuber in 1997, LSTMs have become a pivotal advancement in the field of deep learning, particularly in tasks involving sequential and time-series data.

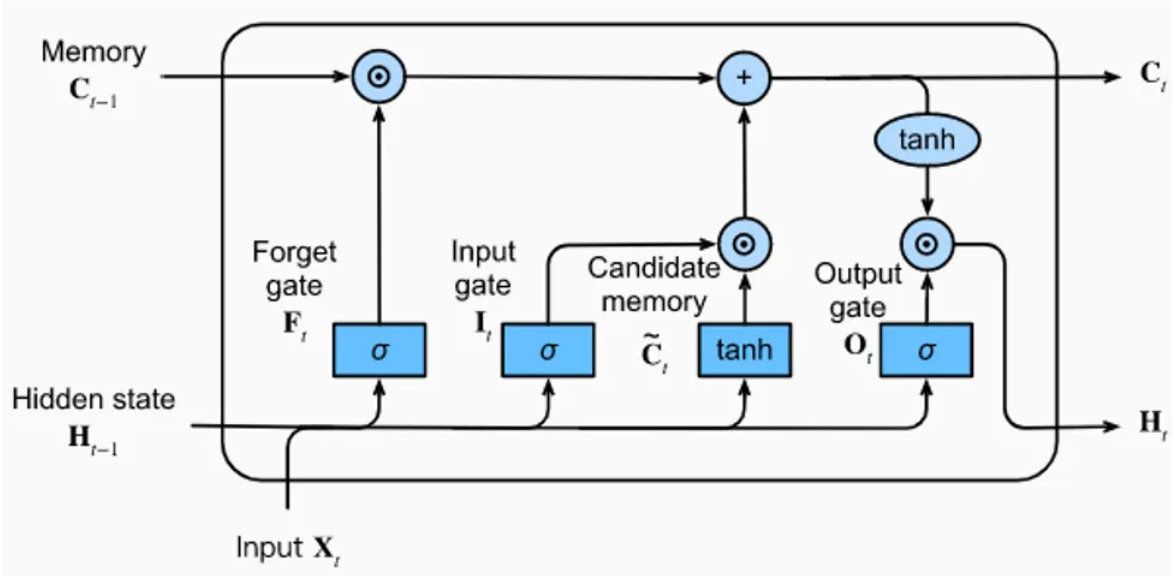

Figure 3: LSTM structure

Components of an LSTM model includes:

1. **Memory Cells:** LSTMs introduce a memory cell that can store information over extended periods, preventing the vanishing gradient problem. This enables LSTMs to capture and retain relevant information from the input sequence.

2. **Gates:** LSTMs utilize three gates—input gate, forget gate, and output gate—to control the flow of information into, out of, and within the memory cell. These gates regulate what information is stored, forgotten, and retrieved, facilitating the learning of long-term dependencies.

3. **Cell State:** The cell state runs along the entire sequence and acts as a conveyer belt, allowing information to be transported across different time steps. The gates help in determining when and how much information is added to or removed from the cell state.

4. **Hidden State:** The hidden state, also known as the output state, is responsible for carrying information to the next time step. It is a filtered version of the cell state, capturing relevant features that contribute to the model's predictions.

In the realm of Natural Language Processing, where understanding sequential dependencies is crucial, LSTMs have found extensive applications as well:

- **Sequence-to-Sequence Models:** LSTMs are integral in sequence-to-sequence models, where they can be employed for tasks like machine translation, text summarization, and question answering, enabling the model to understand and generate coherent sequences.

- **Sentiment Analysis:** Understanding the sentiment of a piece of text often requires capturing nuanced relationships between words. LSTMs excel in this aspect by maintaining a contextual understanding of the input sequence.

- **Language Modeling:** LSTMs are used for language modeling, where the model learns the probabilities of word sequences. This is fundamental in tasks like text generation, speech recognition, and other applications that involve predicting the next element in a sequence.

- **Document Classification:** LSTMs can be applied to tasks such as document classification, where capturing the sequential structure of text is essential for making accurate predictions about the category or topic of a document.

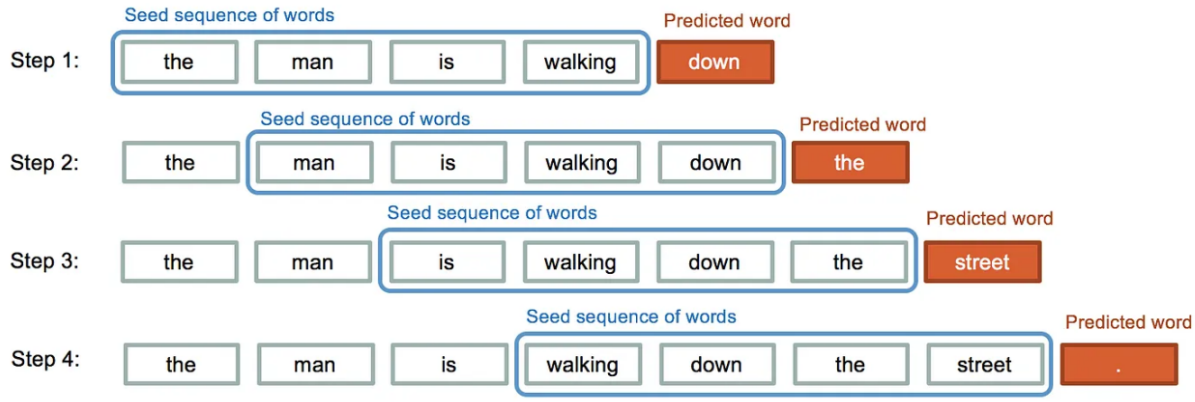

Figure 4: LSTM in NLP

2.3 K-Fold validation

K-fold cross-validation is a method used to assess predictive models by dividing the dataset into k subsets or folds. The model undergoes training and evaluation k times, with a different fold serving as the validation set in each iteration. The average of performance metrics across all folds provides an estimate of the model's generalization performance, aiding in model evaluation, selection, and hyperparameter tuning for more reliable effectiveness.

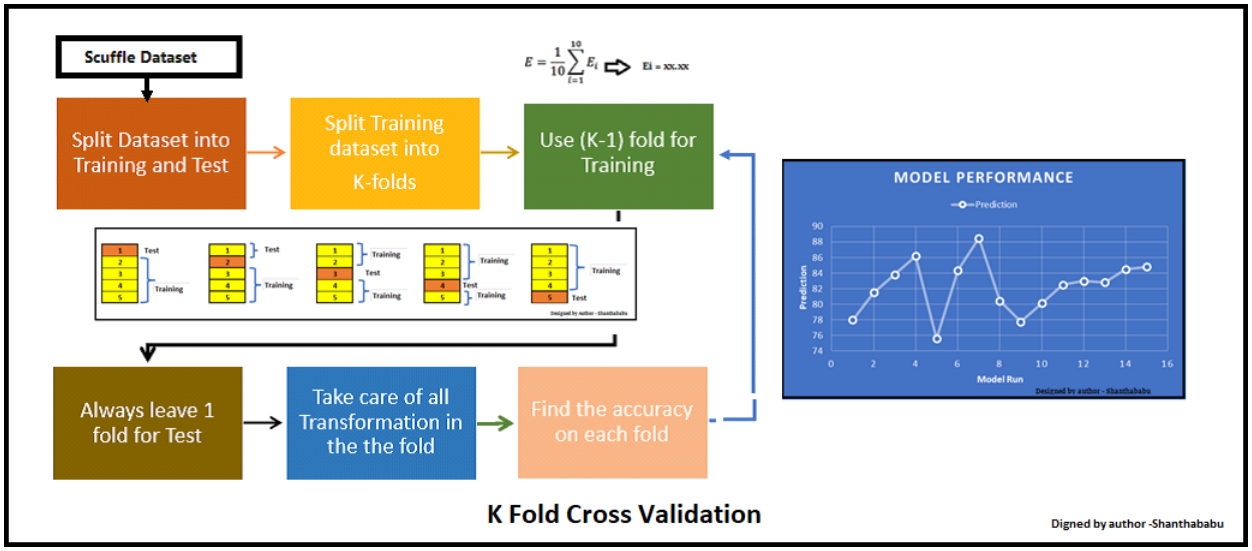

Figure 5: K-Fold validation process

During this process, each fold involves precisely one training and testing cycle, preventing overfitting by ensuring the model is exposed to diverse subsets of the data. This approach addresses the issue of a model performing exceptionally well when trained on the entire dataset, which might lead to limited generalization.

To implement K-Fold Cross Validation, the dataset is typically divided into three sets: Training, Testing, and Validation. The Training and Testing sets support model building and hyperparameter assessment, while the model is validated multiple times based on the specified integer value of K. The dataset is divided into K subsets, and training/testing occurs sequentially K times, ensuring a more generalized model.

2.4 PhoBERT pre-trained model

The cutting-edge language models for the Vietnamese language are the pre-trained PhoBERT models. Specifically, there are two versions available: "base" and "large." These models represent the first publicly

released large-scale monolingual language models tailored for Vietnamese. The name "PhoBERT" is derived from a popular Vietnamese dish.

Figure 6: Pho - a famous dish in Vietnam

- The PhoBERT pre-training methodology is rooted in RoBERTa, a technique that refines the BERT pre-training procedure to enhance overall performance and robustness. Through this approach, PhoBERT has demonstrated superior capabilities compared to earlier monolingual and multilingual methods.
- Across various downstream Vietnamese natural language processing (NLP) tasks, including Part-of-speech tagging, Dependency parsing, Named-entity recognition, and Natural language inference, PhoBERT has set new benchmarks by achieving state-of-the-art performance levels. This highlights the effectiveness of PhoBERT in advancing the field of NLP for the Vietnamese language.

Read more about PhoBERT from here

2.5 RoBERTa: An optimized BERT pretraining model

The RoBERTa model, proposed in the paper "RoBERTa: A Robustly Optimized BERT Pretraining Approach" by Liu et al., is an extension of Google's BERT model introduced in 2018. It enhances BERT by modifying critical hyperparameters, eliminating the next-sentence pretraining objective, and employing larger mini-batches and learning rates during training.

The paper's abstract emphasizes the challenges of comparing different language model pretraining approaches due to the computational expense of training, variations in private datasets, and the significant impact of hyperparameter choices on results. The authors conduct a thorough replication study of BERT pretraining (Devlin et al., 2019), meticulously assessing the influence of various key hyperparameters and training data sizes. The study reveals that BERT was notably undertrained, capable of matching or surpassing the performance of subsequent models. The best RoBERTa model achieves state-of-the-art results on benchmarks such as GLUE, RACE, and SQuAD. These findings underscore the importance of previously overlooked design choices and prompt questions about the origins of recently reported improvements. The authors generously release their models and code for further research.

Read more about the RoBERTa model on huggingface from here.

3 Data

My data is the review from random customers about some product. And there are 3 kinds of reviews: **NEGATIVE** (-1), **NEUTRAL** (0) and **POSITIVE** (1). Which means will will have 3 classes.

- **Training data:** 5100 reviews

```
[65]  data_train
      ✓ 0.0s
```

	Class	Data
0	-1	Mình đã dùng anywhere thế hệ đầu, quả là đầy t...
1	-1	Quan tâm nhất là độ trễ có cao không, dùng thi...
2	-1	dag xài con cùi bắp 98k....pin trâu, mỗi tội đ...
3	-1	logitech chắc hàng phải tiền triệu trở lên dùn...
4	-1	Đang xài con m175 cùi mía , nhà xài nhiều chuộ...
...	...	...
5095	0	Mình mua máy về đc 1 ngày mà điện thoại khác g...
5096	0	Có bạn nào dùng f1w ko.mình dùng m cảm thấy qu...
5097	0	Dùng oppo mà bộ nhớ 4gb thì k chơi games ...
5098	0	Sao tui thích xài hàng oppo mà lựa toàn mấy đứ...
5099	0	mới mở hộp ,oy mở vào camera mà đã có ảnh chụp...

5100 rows × 2 columns

Figure 7: Training set

- **Testing data:** 1050 reviews

```
[91]  data_test
      ✓ 0.0s
```

	Class	Data
0	-1	Nói thiệt là mình thì thì chuột nào mình cũng ...
1	-1	Đang dùng mx1. Cũng ngon nhưng chưa đầy năm mà...
2	-1	Chưa thấy đc điểm thuyết phục để mua, nhất là ...
3	-1	Những phần xem báo tra cứu bản đồ, dịch vụ.. d...
4	-1	ĐÚNG LÀ MUA Ở VIỆT NAM KHÔNG ỨNG DỤNG ĐƯỢC GÌ ...
...	...	...
1045	0	30 củ à :)
1046	0	Apple bán dc thì samsung cũng lời nhiều. Ng...
1047	0	có thể giúp android vượt trội so với ios chớ c...
1048	0	Mẹ mình từng sang Đài Loan và có mua 1 cái iph...
1049	0	Tùng Minh Nguyễn điện thoại của vk bị như này ...

1050 rows × 2 columns

Figure 8: Test set

- **Data distribution in training set:**

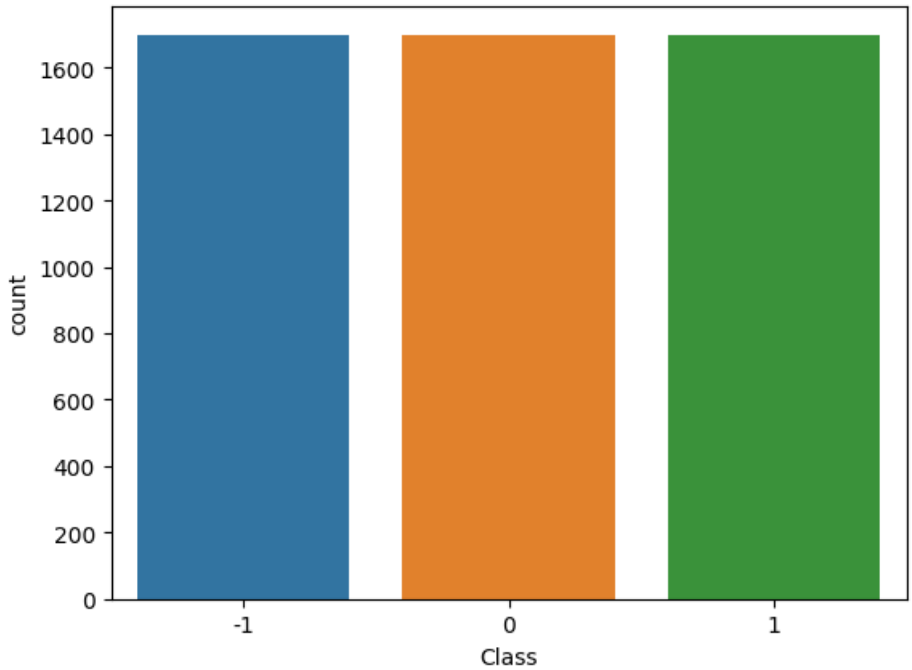

Figure 9: Distribution of classes in the training set

So, data in the train set is very equally distributed.

- **Data preprocessing for CNN and LSTM models:**
  - **Remove digits in each reviews:** Because we are not able to evaluate those digits (even if we think about some encoding methods, those digits are still useless to be transformed to something useful), so the best way is to get rid of them

```python
def process_review(reviews: np.ndarray):
    reviews_processed = []
    for review in reviews:
        review_cool_one = ''.join([char for char in review if char not in digits])
        reviews_processed.append(review_cool_one)

    return reviews_processed

reviews_processed = process_review(reviews)
```

  - **One hot encoding:** I created encoded labels with following rules:
    * If `labels == -1` (**NEGATIVE**), I encode it into $\begin{bmatrix}0\\0\\1\end{bmatrix}$
    * If `labels == 0` (**NEUTRAL**), I encode it into $\begin{bmatrix}0\\1\\0\end{bmatrix}$
    * If `labels == 1` (**POSITIVE**), I encode it into $\begin{bmatrix}1\\0\\0\end{bmatrix}$

```python
def oneHotEncode(labels):
    encoded_labels = []
    for label in labels:
        if label == -1:
            encoded_labels.append([1,0,0])
        elif label == 0:
            encoded_labels.append([0,1,0])
        else:
            encoded_labels.append([0,0,1])

    return np.array(encoded_labels)

encoded_labels = oneHotEncode(labels)

print(encoded_labels.shape)
```

- **Tokenization:** I use the `ViTokenizer` library to tokenize each reviews and append the the words into a list.

```python
def tokenize(reviews_processed: list):
    word_reviews = []
    for review in reviews_processed:
        review = ViTokenizer.tokenize(review.lower())
        word_reviews.append(review.split())

    return word_reviews

word_reviews = tokenize(reviews_processed)
```

- Then, we start transforming our `word_reviews` into sequences of integer for training. I defined maximum number of unique words to use is 10000 (i.e. `MAX_VOCAB_SIZE = 10000`). Maximum number of words we use in a review will be 300 (which means `MAX_SEQUENCE_LENGTH = 300`)

```python
tokenizer = Tokenizer(num_words=MAX_VOCAB_SIZE, lower=True,
    char_level=False)
tokenizer.fit_on_texts(word_reviews)
sequences_train = tokenizer.texts_to_sequences(word_reviews
   )
word_index = tokenizer.word_index
data = pad_sequences(sequences_train, maxlen=
   MAX_SEQUENCE_LENGTH)
labels = encoded_labels
```

- **Create embedding matrix:** I started by loading pre-trained word vectors stored in file `vi-model-CBOW.bin`. These vectors are typically generated by algorithms like Word2Vec or GloVe, which can capture semantic relationships between words. Then, I created an embedding matrix that will be used to initialize the embedding layer. Each word in the vocabulary gets a vector in this matrix. If a word is present in the pre-trained model, its pre-trained vector is used; otherwise, a random vector is generated. Finally, an embedding layer is created using the `Keras` library, with the embedding matrix as initial weights. This layer can be added to

a neural network model and is set to be trainable, allowing the model to fine-tune the word vectors during training.

```python
def create_embedding():
    # load from pre-trained model

    word_vectors = KeyedVectors.load_word2vec_format('vi-model-CBOW.bin', binary=True)
    vocabulary_size = min(len(word_index)+1,MAX_VOCAB_SIZE)
    embedding_matrix = np.zeros((vocabulary_size, EMBEDDING_DIM))
    print("vocab size", vocabulary_size)
    for word, i in word_index.items():
        if i>=MAX_VOCAB_SIZE:
            continue
        try:
            embedding_vector = word_vectors[word]
            embedding_matrix[i] = embedding_vector
        except KeyError:
            embedding_matrix[i]=np.random.normal(0,np.sqrt(0.25),EMBEDDING_DIM)

    del(word_vectors)

    from keras.layers import Embedding
    embedding_layer = Embedding(vocabulary_size,
                                EMBEDDING_DIM,
                                weights=[embedding_matrix],
                                trainable=True)

    return embedding_layer

embedding_layer = create_embedding()
```

4 Structure

4.1 Using only CNN

4.1.1 Define structure and arguments

My proposed CNN structure is as follow:

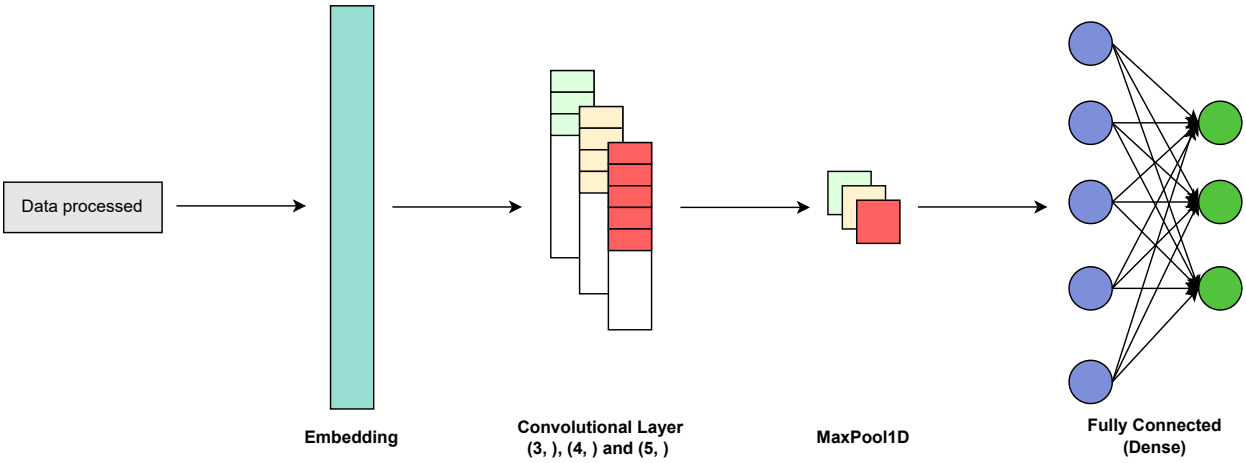

Figure 10: CNN-structured model

More informations about the model:

Table 1: A CNN Model

Layer (type)	**Output Shape**	**Parameter #**
input_2 (InputLayer)	(None, 300)	0
embedding (Embedding)	(None, 300, 400)	3167600
reshape_1 (Reshape)	(None, 300, 400)	0
conv1d (Conv1D)	(None, 298, 100)	120100
conv1d_1 (Conv1D)	(None, 297, 100)	160100
conv1d_2 (Conv1D)	(None, 296, 100)	200100
max_pooling1d (MaxPooling1D)	(None, 1, 100)	0
max_pooling1d_1 (MaxPooling1D)	(None, 1, 100)	0
max_pooling1d_2 (MaxPooling1D)	(None, 1, 100)	0
reshape_2 (Reshape)	(None, 1, 100)	0
reshape_3 (Reshape)	(None, 1, 100)	0
reshape_4 (Reshape)	(None, 1, 100)	0
concatenate (Concatenate)	(None, 1, 300)	0
flatten (Flatten)	(None, 300)	0
dropout_1 (Dropout)	(None, 300)	0
dense_1 (Dense)	(None, 3)	903
Total params: 3648803 (13.92 MB) **Trainable params: 3648803 (13.92 MB)** **Non-trainable params: 0 (0.00 Byte)**		

I also define the early stopping with `min_delta = 0.001`, `patience = 4` as follow:

```
early_stopping = EarlyStopping(monitor='val_loss', min_delta
    =0.001, patience=4, verbose=1)
callbacks_list = [early_stopping]
```

4.1.2 Train on training set

The training process contains 20 epochs with batch size is 256:

```
model.fit(data, labels, validation_split=0.2, epochs=20,
    batch_size=256, callbacks=callbacks_list, shuffle=True)
```

After training, the accuracy measured on the training set is 96.05%, loss is 0.6528

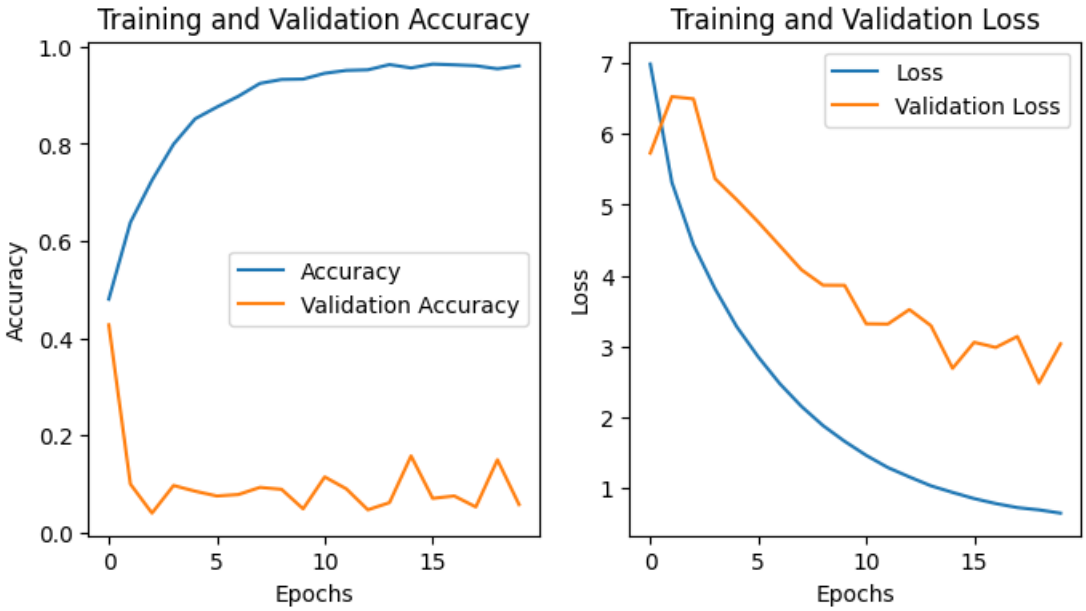

Figure 11: Training and validation accuracy and loss

4.1.3 Evaluation on test set

After loading the test set and pass through data-preprocessing phase:

```
score = model.evaluate(data_test, labels_test)
```

We get the accuracy of the model is about 61.90%, loss is 1.46
Confusion matrix:

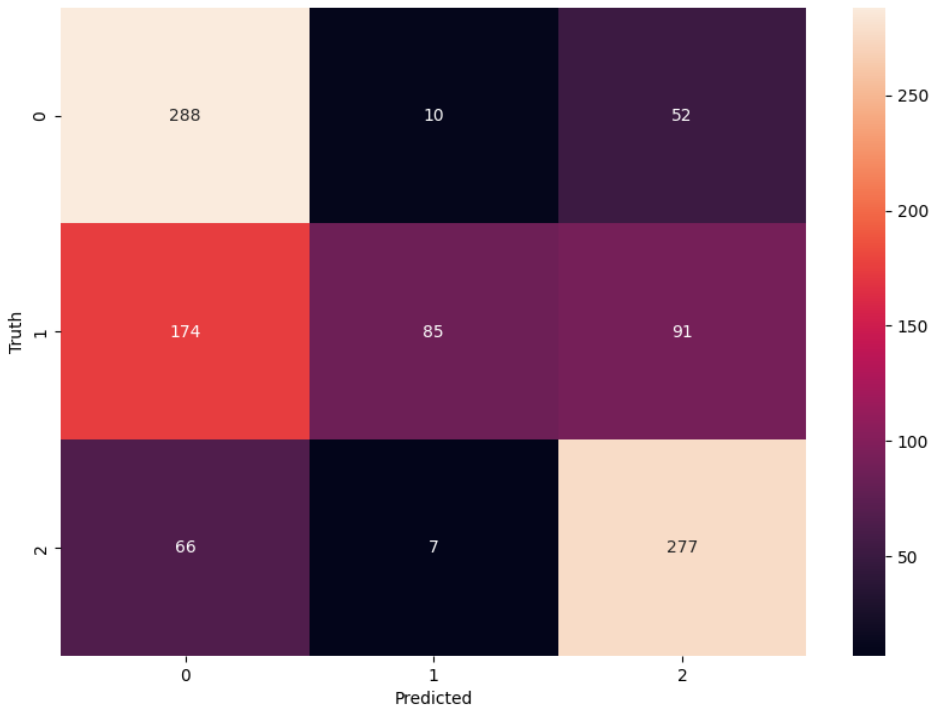

Figure 12: Confusion matrix for CNN model

I also measure some other metrics such as recall, precision and the F-Score.

Table 2: Model Evaluation Metrics

Model	Accuracy	Recall	Precision	F-Score
CNN	0.619048	0.619048	0.679437	0.583874

4.2 Using only LSTM

4.2.1 Defined structure and arguments

My proposed structure of LSTM is to use a Bidirectional LSTM model combining with a fully connected layer as follow:

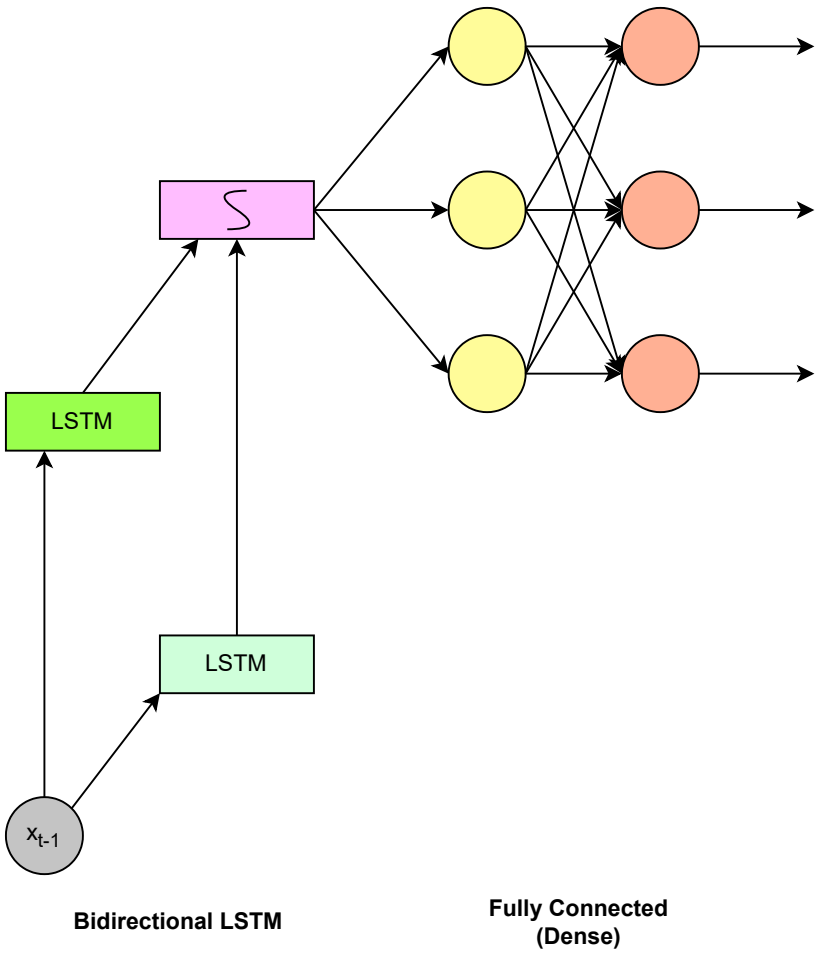

Figure 13: LSTM model

Model overview:

Table 3: LSTM Model

Layer (type)	Output Shape	Parameter #
input_1 (InputLayer)	(None, 300)	0
embedding (Embedding)	(None, 300, 400)	3167600
reshape (Reshape)	(None, 300, 400)	0
bidirectional (Bidirectional)	(None, 2048)	11673600
dropout (Dropout)	(None, 2048)	0
dense (Dense)	(None, 3)	6147
Total params: 14847347 (56.64 MB) **Trainable params: 14847347 (56.64 MB)** **Non-trainable params: 0 (0.00 Byte)**		

Define early stopping:

```
#define callbacks
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    patience=4, verbose=1)
callbacks_list = [early_stopping]

lstm_model.fit(data, labels, validation_split=0.2, epochs=20,
   batch_size=256, callbacks=callbacks_list, shuffle=True)
```

4.2.2 Train on training set

```
model.fit(data, labels, validation_split=0.2, epochs=20,
   batch_size=256, callbacks=callbacks_list, shuffle=True)
```

After training, the accuracy measured on the training set is 98.41%, loss is 0.1085. Early stopping at epoch 10th

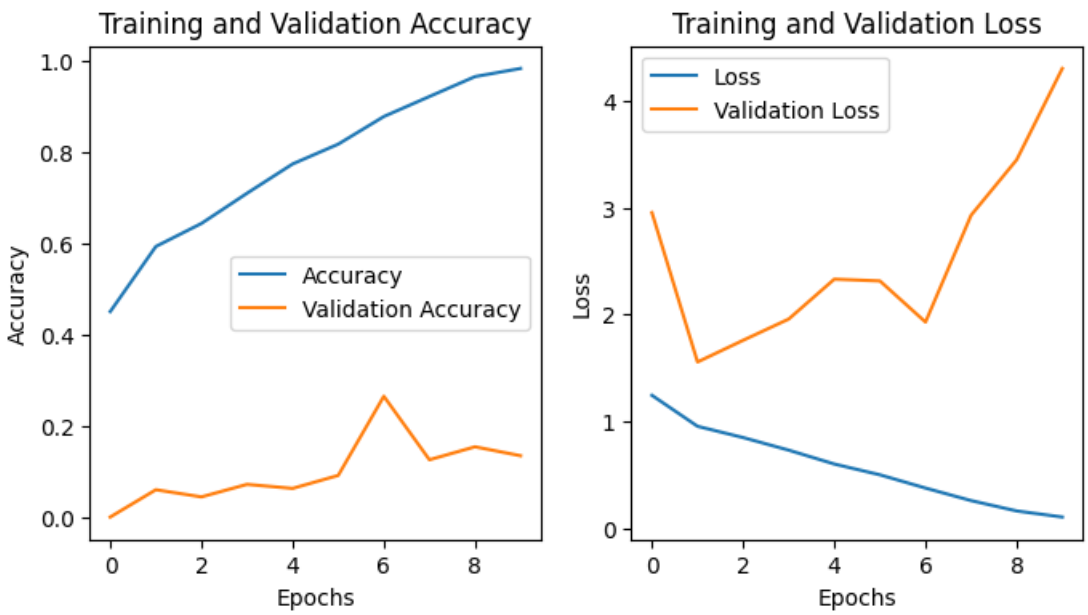

Figure 14: Training result

4.2.3 Evaluation on test set

After loading the test set and pass through data-preprocessing phase:

```
score = model.evaluate(data_test, labels_test)
```

We get the accuracy of the model is about 63.33%, loss is 1.58.
Confusion matrix:

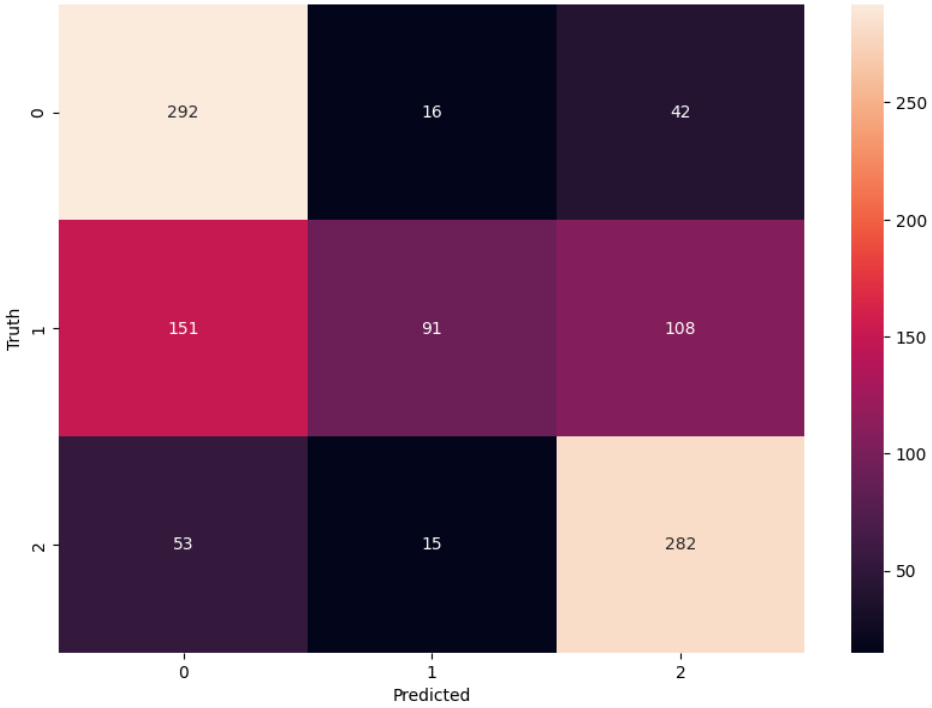

Figure 15: Confusion matrix for LSTM model

Other metrics:

Model	Accuracy	Recall	Precision	F-Score
LSTM	0.633333	0.633333	0.662463	0.599043

Table 4: Model Evaluation Metrics

4.3 Using CNN & LSTM combination

4.3.1 CNN layers before LSTM layers

Defined structure and arguments

A combination between LSTM layers and CNN layers, with LSTM layers are set before passing through the CNN layers:

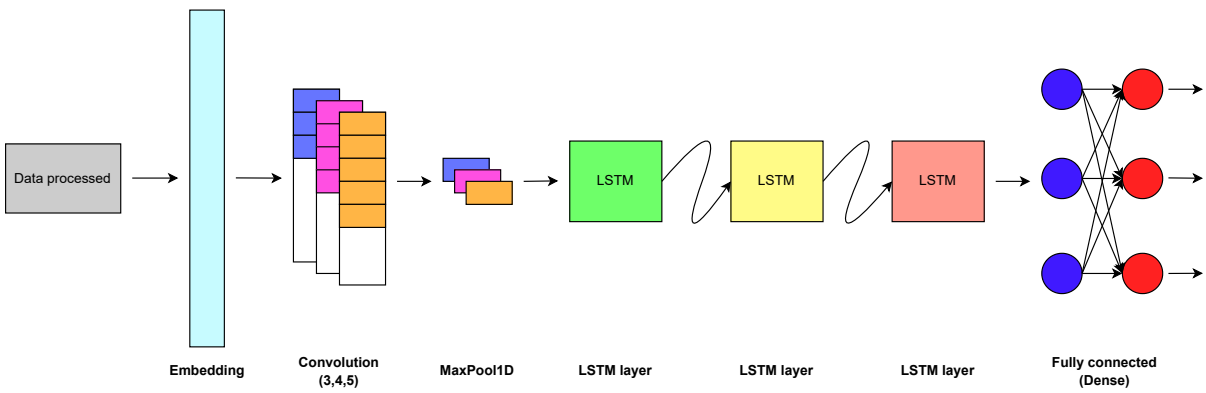

Figure 16: CNN-LSTM model

Model overview:

Table 5: CRNN Model

Layer (type)	**Output Shape**	**Parameter #**
input_3 (InputLayer)	(None, 300)	0
embedding (Embedding)	(None, 300, 400)	3167600
reshape_5 (Reshape)	(None, 300, 400)	0
conv1d_3 (Conv1D)	(None, 300, 100)	120100
conv1d_4 (Conv1D)	(None, 300, 100)	160100
conv1d_5 (Conv1D)	(None, 300, 100)	200100
max_pooling1d_3 (MaxPooling1D)	(None, 1, 100)	0
max_pooling1d_4 (MaxPooling1D)	(None, 1, 100)	0
max_pooling1d_5 (MaxPooling1D)	(None, 1, 100)	0
reshape_6 (Reshape)	(None, 1, 100)	0
reshape_7 (Reshape)	(None, 1, 100)	0
reshape_8 (Reshape)	(None, 1, 100)	0
concatenate_1 (Concatenate)	(None, 1, 300)	0
lstm_4 (LSTM)	(None, 1, 1024)	5427200
lstm_5 (LSTM)	(None, 1, 512)	3147776
lstm_6 (LSTM)	(None, 256)	787456
dropout_2 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 3)	771
Total params: 13011103 (49.63 MB) **Trainable params: 13011103 (49.63 MB)** **Non-trainable params: 0 (0.00 Byte)**		

Train on training set

```
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    patience=8, verbose=1)
callbacks_list = [early_stopping]

cnn_lstm_model.fit(data, labels, validation_split=0.2, epochs=20,
   batch_size=256, callbacks=callbacks_list, shuffle=True)
```

After training, the accuracy measured on the training set is 98.82%, loss is 0.3427. Early stopping at epoch 14th

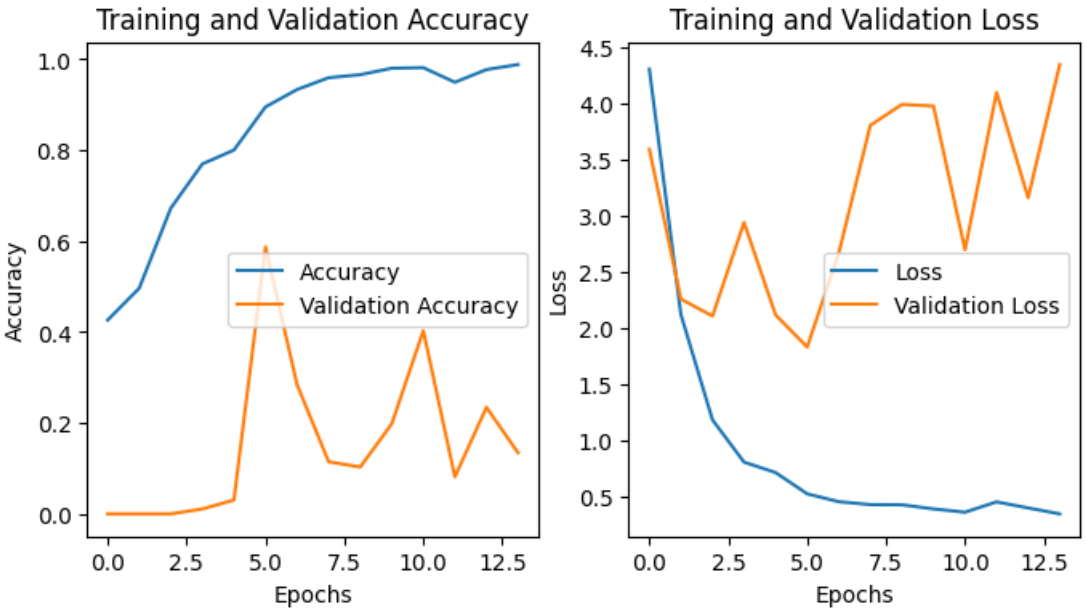

Figure 17: CNN-LSTM training result

Evaluation on test set

After loading the test set and pass through data-preprocessing phase:

```
score = model.evaluate(data_test, labels_test)
```

We get the accuracy of the model is about 65.62%, loss is 1.83

Confusion matrix:

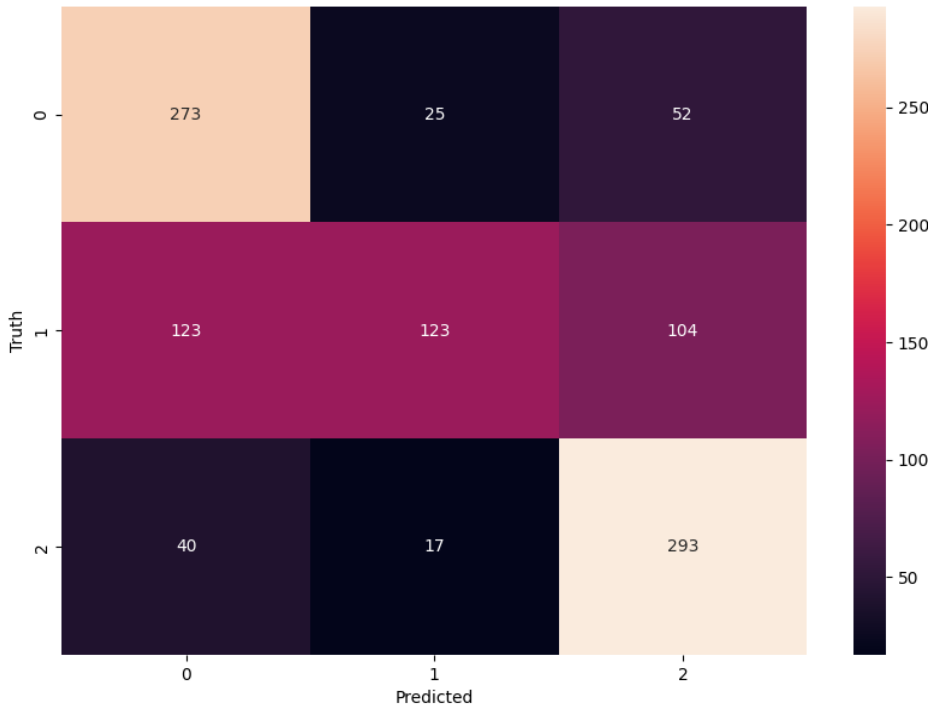

Figure 18: Confusion matrix for CNN-LSTM model

Other metrics:

Model	Accuracy	Recall	Precision	F-Score
CRNN	0.65619	0.65619	0.674721	0.635248

Table 6: Model Evaluation Metrics

4.3.2 LSTM layers before CNN layers

Defined structure and arguments

A combination between LSTM layers and CNN layers, with LSTM layers are set before passing through the CNN layers:

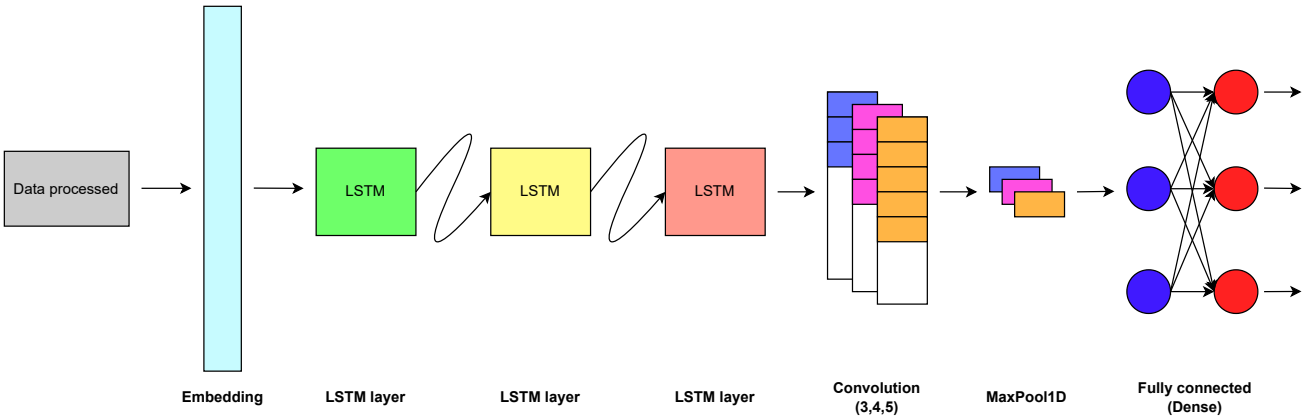

Figure 19: LSTM-CNN model

Model overview:

Table 7: RCNN Model

Layer (type)	Output Shape	Parameter #
input_2 (InputLayer)	(None, 300)	0
embedding (Embedding)	(None, 300, 400)	3167600
reshape_1 (Reshape)	(None, 300, 400)	0
lstm_1 (LSTM)	(None, 300, 1024)	5836800
lstm_2 (LSTM)	(None, 300, 512)	3147776
lstm_3 (LSTM)	(None, 300, 256)	787456
conv1d (Conv1D)	(None, 300, 100)	76900
conv1d_1 (Conv1D)	(None, 300, 100)	102500
conv1d_2 (Conv1D)	(None, 300, 100)	128100
max_pooling1d (MaxPooling1D)	(None, 1, 100)	0
max_pooling1d_1 (MaxPooling1D)	(None, 1, 100)	0
max_pooling1d_2 (MaxPooling1D)	(None, 1, 100)	0
reshape_2 (Reshape)	(None, 1, 100)	0
reshape_3 (Reshape)	(None, 1, 100)	0
reshape_4 (Reshape)	(None, 1, 100)	0
concatenate (Concatenate)	(None, 1, 300)	0
flatten (Flatten)	(None, 300)	0
dropout_1 (Dropout)	(None, 300)	0
dense_1 (Dense)	(None, 3)	903
Total params: 13248035 (50.54 MB) **Trainable params: 13248035 (50.54 MB)** **Non-trainable params: 0 (0.00 Byte)**		

Train on training set

```
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    patience=8, verbose=1)
callbacks_list = [early_stopping]

lstm_cnn_model.fit(data, labels, validation_split=0.2, epochs=20,
   batch_size=256, callbacks=callbacks_list, shuffle=True)
```

After training, the accuracy measured on the training set is 98.33%, loss is 0.1925. Early stopping at epoch 11th

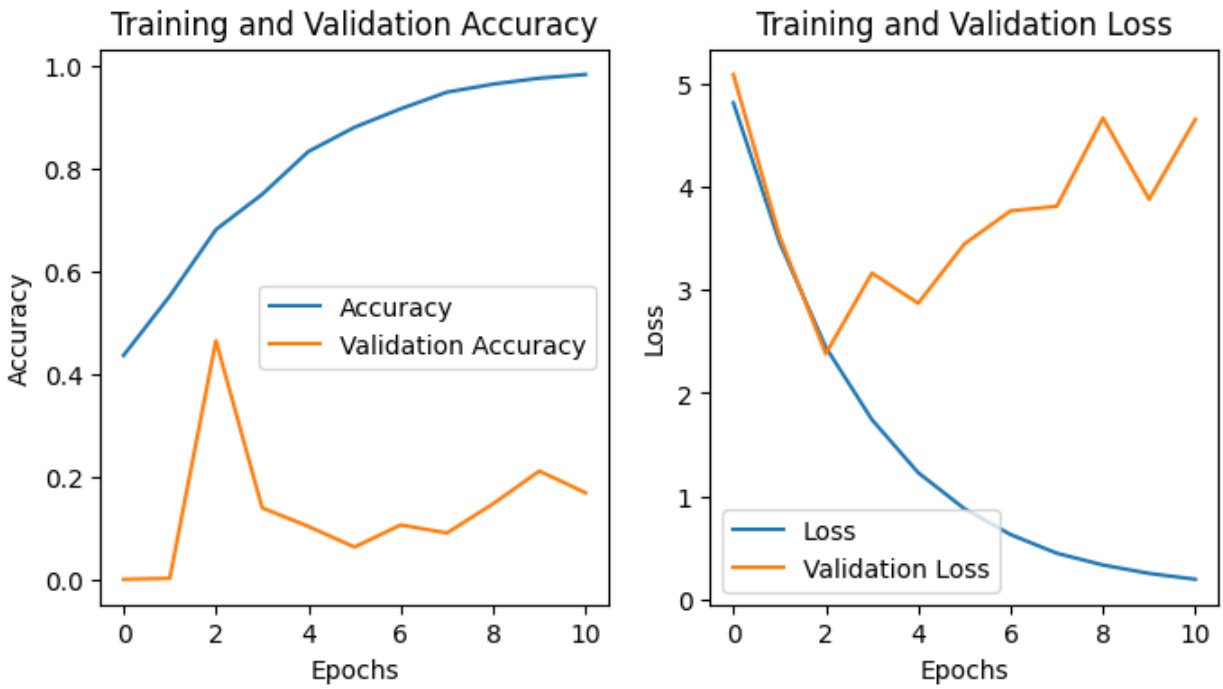

Figure 20: LSTM-CNN training result

Evaluation on test set

After loading the test set and pass through data-preprocessing phase:

```
score = model.evaluate(data_test, labels_test)
```

We get the accuracy of the model is about 64.29%, loss is 1.82.
Confusion matrix:

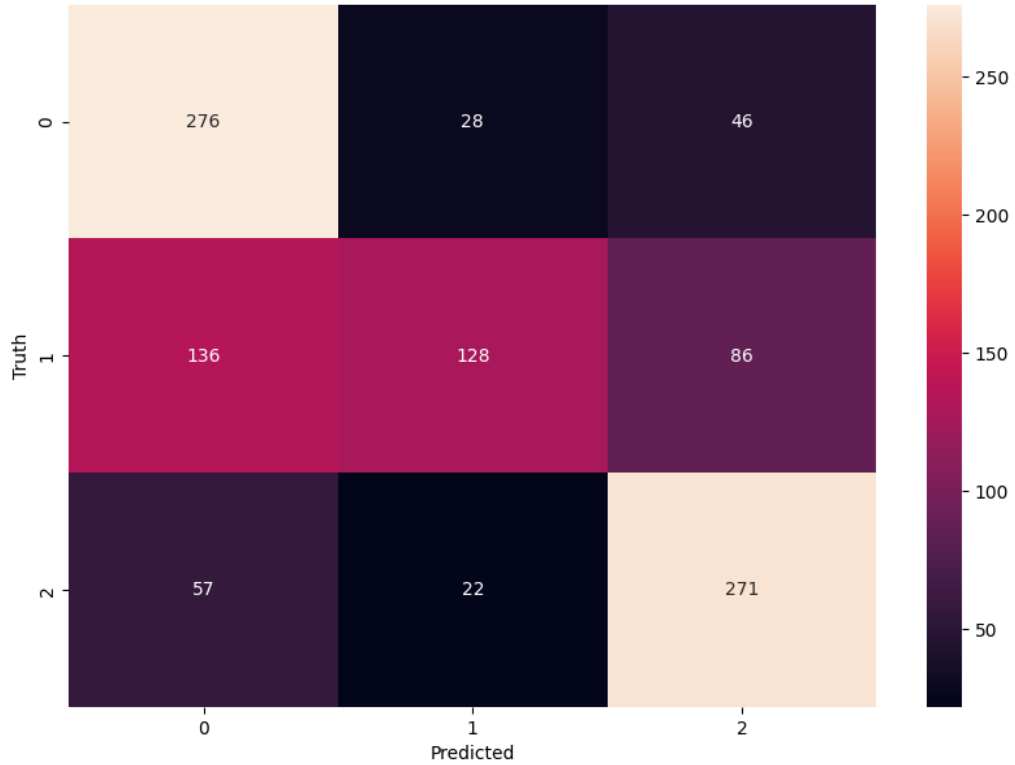

Figure 21: Confusion matrix for LSTM-CNN model

Other metrics:

Model	Accuracy	Recall	Precision	F-Score
RCNN	0.642857	0.642857	0.660015	0.62621

Table 8: Model Evaluation Metrics

4.4 Applying K-Fold with PhoBERT

4.4.1 Data preprocessing and folding data

First, I changed the label of the classes a little bit, from -1 (**NEGATIVE**), 0 (**NEUTRAL**) and 1 (**POSITIVE**) to $0, 1$ and 2, respectively. The reason for this will be explained later:

```
train_df['Class'] = train_df['Class'].map({-1:0, 0:1, 1:2})
test_df['Class'] = test_df['Class'].map({-1:0, 0:1, 1:2})
```

Then, we use Stratified K-Fold from `sklearn` with number of split is 5:

```
N_SPLITS = 5
skf = StratifiedKFold(n_splits=N_SPLITS)
for fold, (_, val_) in enumerate(skf.split(X=train_df, y=train_df.
    Class)):
    train_df.loc[val_, "kfold"] = fold
```

	Class	Data	kfold
3863	1	Chả biết có tốt không cơ mà xài mãi chả thấy l...	1.0
3448	1	không có chút cảm giác lag à????? Bực mình thậ...	0.0
4735	1	Mua thì nên mua những máy có đồ thay dễ dàng.....	3.0
1749	2	Hoan toan hai long voi mot trai nghiem chua tu...	0.0
3603	1	Bản rẻ nhất Apple Watch Sport 38mm, giá tầm 9t...	0.0

Figure 22: Data after K-Fold

To use PhoBERT, we need a tokenizer. I will use the tokenizer from VinAI PhoBERT-based:

```
tokenizer = AutoTokenizer.from_pretrained("vinai/phobert-base",
    use_fast=False)
```

The encoding will be used from the object `encode_plus` from our tokenizer. I also set the return tensors so that I can use GPU for training:

```
encoding = self.tokenizer.encode_plus(
    text,
    truncation=True,
    add_special_tokens=True,
    max_length=self.max_len,
    padding='max_length',
    return_attention_mask=True,
    return_token_type_ids=False,
    return_tensors='pt',
)
```

Next, I visualize the distribution of the length between token of sentences:

```
# Distribution of length of Sentence
all_data = train_df.Data.tolist() + test_df.Data.tolist()
all_data = [' '.join(simple_preprocess(text)) for text in all_data
    ]
encoded_text = [tokenizer.encode(text, add_special_tokens=True)
    for text in all_data]
token_lens = [len(text) for text in encoded_text]
sns.displot(token_lens)
plt.xlim([0,max(token_lens)])
plt.xlabel('Token Count')
```

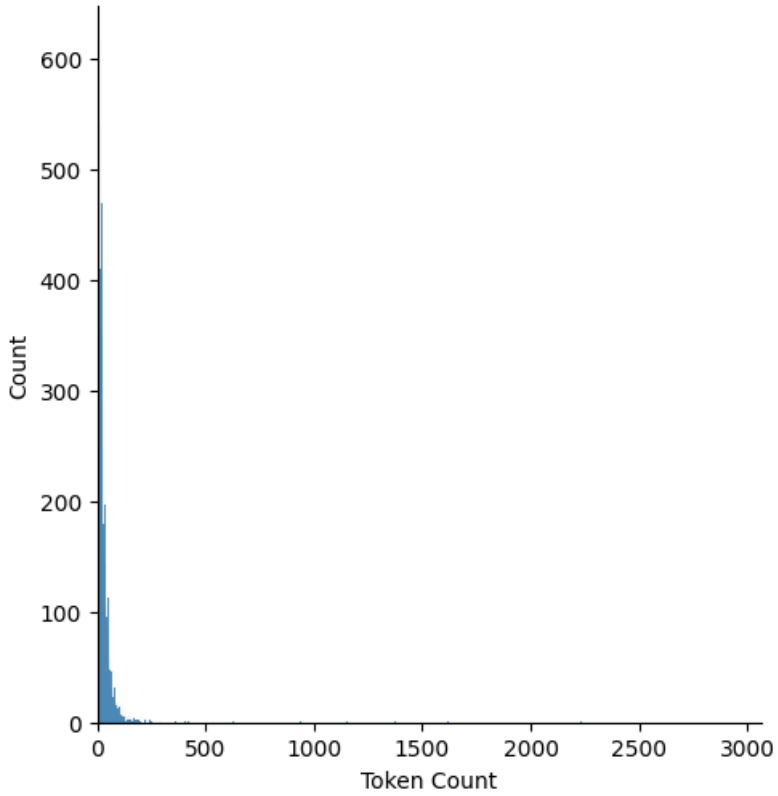

Figure 23: Token distribution between sentences

We can see that most of token counts lies on the left side of total distribution, which means most of sentences are from 0 to 250 tokens. This suggest that we should take the first 200 tokens for analysis.

4.4.2 Training on train set

Now, I will create a classifier class, in which i will initial PhoBERT, Dropout layer, Linear transformation to incoming data and some weights as follow:

```python
class SentimentClassifier(nn.Module):
    def __init__(self, n_classes):
        super(SentimentClassifier, self).__init__()
        self.bert = AutoModel.from_pretrained("vinai/phobert-base")
        self.drop = nn.Dropout(p=0.3)
        self.fc = nn.Linear(self.bert.config.hidden_size, n_classes)
        nn.init.normal_(self.fc.weight, std=0.02)
        nn.init.normal_(self.fc.bias, 0)

    def forward(self, input_ids, attention_mask):
        last_hidden_state, output = self.bert(
            input_ids=input_ids,
            attention_mask=attention_mask,
            return_dict=False
        )

        x = self.drop(output)
        x = self.fc(x)
        return x
```

I also use **Adam optimizer**, the loss function is **Cross entropy loss**.
In total there are 6 epochs, each with 6 folds. Each fold reaches an average accuracy of 0.89

4.4.3 Evaluate on test set

We obtained the following table after evaluating on the test set:

Table 9: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.72	0.68	0.70	350
1	0.62	0.65	0.64	350
2	0.77	0.78	0.78	350
Accuracy			0.71	1050
Macro Avg	0.71	0.71	0.71	1050
Weighted Avg	0.71	0.71	0.71	1050

4.5 RoBERTa combining with LSTM (in developing mode)

4.5.1 Defined structure

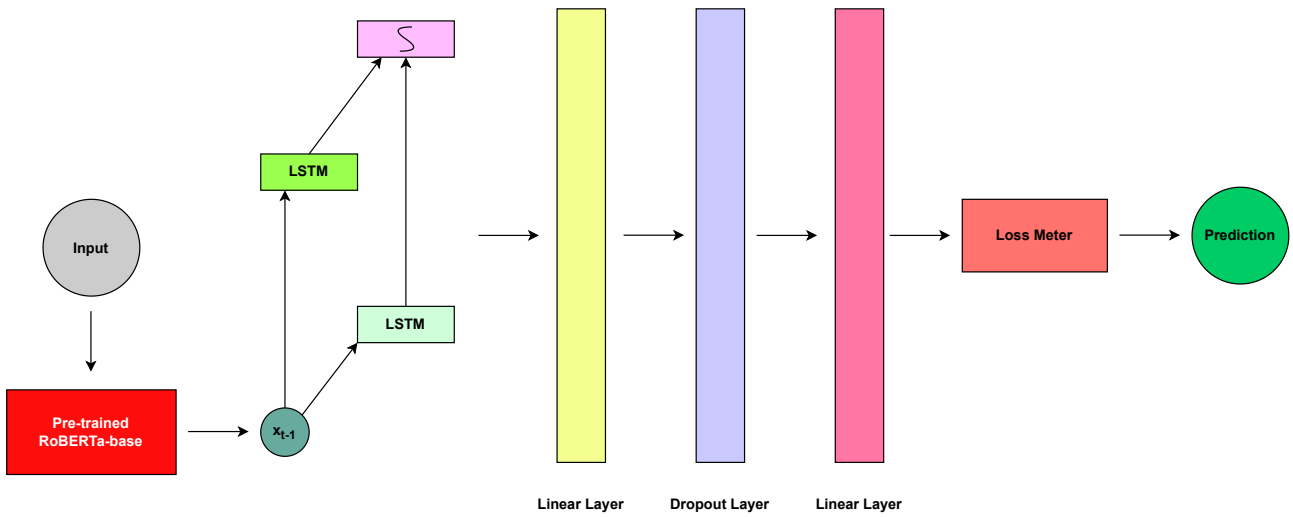

Figure 24: RoBERTa and LSTM defined structure

The model has a hierarchical structure, combining the RoBERTa transformer-based architecture with a custom classification head.

1. **Base RoBERTa Model** (`RobertaForSequenceClassification`):
   - The model starts with a pre-trained RoBERTa base, which is a transformer-based architecture specifically designed for sequence classification tasks.
   - The RoBERTa model processes the input sequence and produces a contextualized representation for each token in the sequence.
2. **Custom Classification Head** (`SentimentRobertaClassificationHead`):
   - A custom classification head is added on top of the RoBERTa base to perform task-specific processing.
   - The custom head includes an LSTM layer with bidirectionality and two layers, followed by a linear layer, a tanh activation, a dropout layer, and another linear layer.
   - This head is designed for sentence-level classification tasks, such as sentiment analysis.
3. **Forward Pass**:
   - During the forward pass, input tokens are fed into the RoBERTa base, and the output is obtained, which is a sequence of contextualized representations for each token (`sequence_output`).
   - The `sequence_output` is then passed through the custom classification head, producing the final logits for classification.
4. **Loss Calculation:** Depending on the problem type specified in the model's configuration, the loss is calculated using different loss functions. For single-label classification, cross-entropy loss is used (`CrossEntropyLoss`).
5. **Output**: The model returns a `SequenceClassifierOutput` object, which includes the loss, logits, hidden states, and attentions.

4.5.2 Trainer

1. **Initialization**:
   - The trainer is initialized with a configuration (`SentimentConfig`), which includes settings for the model, training, and evaluation.
   - It loads a pre-trained sentiment analysis model (`SentimentModel`) and sets the device for training.
2. **Dataset loading**:
   - The trainer loads the training and evaluation datasets using the `SentimentDataset` class, which is responsible for tokenizing and formatting the data.
3. **Collator function**:
   - The `collator` function is defined to organize batches of data during training. It stacks the values for each key in the batch, converting them to PyTorch tensors, and placing them on the specified training device.

4.5.3 Configuring environment:

- **Metrics initialization**: Evaluation metrics (precision, recall, F1, and accuracy) are loaded using functions from an external `evaluate` module and stored in the `metrics` dictionary.
- **Metrics computation**: The `compute_metrics` function is defined to compute evaluation metrics based on model predictions and ground truth labels. It uses the loaded metric functions and calculates precision, recall, F1, and accuracy.
- **Trainer initialization**: Configures training arguments, including output directory, evaluation strategy, batch size, learning rate, etc. The `Trainer` object is initialized with the pre-trained model, training and evaluation datasets, tokenizer, collator function, and the metric computation function.

4.5.4 Important parameters

Default Values:

- `pretrained_path`: "vinai/phobert-base-v2"
- `training_learning_rate`: 1×10^{-5}
- `training_weight_decay`: 0.01
- `training_batch_size`: 16
- `training_num_epochs`: 10
- `training_metrics`: "f1"
- `model_input_max_length`: 200
- `model_num_labels`: 3

4.5.5 Training result

Table 10: Training Process Metrics

Step	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
50	0.682400	0.656380	0.441638	0.382324	0.309005	0.382324
100	0.640900	0.638077	0.211604	0.325720	0.193526	0.325720
150	0.638200	0.637409	0.579558	0.342602	0.241726	0.342602
200	0.636600	0.633731	0.360749	0.463754	0.368330	0.463754
250	0.623100	0.611624	0.537211	0.500497	0.465276	0.500497
300	0.594600	0.580077	0.545409	0.551142	0.499169	0.551142
350	0.549000	0.538223	0.611530	0.613704	0.598715	0.613704
400	0.517400	0.513331	0.636816	0.630586	0.623883	0.630586
450	0.481200	0.497214	0.670162	0.647468	0.617116	0.647468
500	0.463100	0.476585	0.697219	0.680238	0.653537	0.680238
550	0.417800	0.471310	0.692136	0.694141	0.680882	0.694141
600	0.398700	0.459995	0.712989	0.714995	0.713744	0.714995
650	0.391500	0.464270	0.701938	0.698113	0.683536	0.698113
700	0.374900	0.447809	0.708550	0.712016	0.708952	0.712016
750	0.349900	0.442706	0.720661	0.721946	0.720793	0.721946
800	0.352200	0.438353	0.722608	0.719960	0.720704	0.719960
850	0.325900	0.441691	0.717379	0.718967	0.717895	0.718967
900	0.319200	0.440729	0.716516	0.718967	0.717205	0.718967
950	0.315200	0.433088	0.733795	0.729891	0.731062	0.729891
1000	0.308500	0.437416	0.720654	0.720953	0.720474	0.720953
1050	0.324800	0.441484	0.711957	0.714995	0.712804	0.714995
1100	0.301700	0.441112	0.707742	0.711023	0.708699	0.711023
1150	0.305500	0.439622	0.714570	0.716981	0.715330	0.716981
1200	0.297900	0.441865	0.715761	0.718967	0.716510	0.718967
1250	0.293600	0.441559	0.715938	0.718967	0.716693	0.718967

As we can see, the accuracy is around 0.72, which makes a whole difference with methods represented earlier.

4.5.6 Testing result

After testing, the accuracy of the model is 0.3447

5 Result and analysis

After experimenting with different models and tuning some hyper-parameters, we have the below table:

Model	**Accuracy on test set**
PhoBERT-based Model	0.77
CNN+LSTM Model	0.65619
LSTM+CNN Model	0.642857
LSTM Model	0.633333
CNN Model	0.619048
RoBERTa + LSTM (developing)	0.3447

- The LSTM model achieves a moderate accuracy on the test set. It demonstrates a reasonable ability to understand and classify the given data.
- The CNN model performs slightly below the LSTM model in terms of accuracy. The CNN model may have strengths in capturing certain patterns, but its overall performance is comparable to the LSTM model.
- The combination of LSTM and CNN shows a slight improvement in accuracy compared to individual models. This hybrid approach may benefit from the strengths of both LSTM and CNN architectures, potentially capturing a wider range of features.
- The order of the LSTM and CNN combination seems to impact the performance, with CNN followed by LSTM achieving higher accuracy. The sequence of model architectures can play a crucial role, and in this case, starting with CNN before LSTM appears to yield better results.
- The PhoBERT-based model stands out with a significantly higher accuracy. PhoBERT, a pre-trained transformer model for Vietnamese, demonstrates strong language understanding capabilities and outperforms traditional architectures.
- The RoBERTa + LSTM model is still under development and exhibits lower accuracy. Further refinement and training are likely required to improve the model's performance. Consider revisiting hyperparameters, dataset augmentation, or fine-tuning.

Overall, The PhoBERT-based model outperforms the other architectures, emphasizing the importance of leveraging pre-trained models tailored to specific languages. Hybrid models combining LSTM and CNN show potential, with the order of these components affecting performance. The RoBERTa + LSTM model is in the development stage, and improvements are expected with additional tuning and training.

6 Conclusion

The experimental results reveal insightful trends in the performance of various sentiment analysis models. The PhoBERT-based model emerged as the top performer, showcasing its remarkable efficacy in capturing language intricacies specific to Vietnamese, yielding an impressive accuracy of 0.77 on the test set. Notably, the order of components in hybrid models combining LSTM and CNN architectures proved influential, with the CNN+LSTM model outperforming its counterpart (LSTM+CNN). Traditional models, such as standalone LSTM and CNN, demonstrated moderate accuracy, being surpassed by more advanced counterparts. The RoBERTa + LSTM model, still in development, exhibited lower accuracy (0.3447), indicating the need for further refinement and training. Considering the diverse performance across models, there's potential for improvement through ensemble methods, combining predictions from multiple models. These conclusions suggest that leveraging pre-trained models tailored to the target language, optimizing architecture sequences, and exploring ensemble techniques are critical considerations for developing robust sentiment analysis models in this context.

7 Reference

Here is my Google Colab link in case you want to review my code clearer: https://colab.research.google.com/drive/1ufjktcpnS0xFNPwgoJkWbmIbwRDq79ts?usp=sharing

Data link:

- Train set: https://drive.google.com/file/d/1_u28F7i1ZxagF81mpppEvuKZBRfksnnw/view?usp=drive_link
- Test set: https://drive.google.com/file/d/1ZtGNRjvkjF8g-pkijb-JHZ6cuhi144ur/view?usp=drive_link